

LAPORAN TUGAS AKHIR UAS

Perancangan dan Implementasi Sistem Robotika Berbasis Intelligent Systems



Disusun Oleh (Kelompok UAS):

Nama	NIM
Muhammad Khalis Farhan Azvi	— 22/493199/TK/54016
Valen Fatli	— 22/493755/TK/54109
Yasir Abdulaziz	— 22/497079/TK/54468
Galih Salman Sadewo	— 22/498121/TK/54625
Faris Afif Ridyasmara	— 22/502798/TK/54902

Dosen Pengampu: Dr.Eng. Dwi Joko Suroso, S.T., M.Eng.

Instrumentasi Sistem Robotika

Departemen Teknik Nuklir dan Teknik Fisika

Fakultas Teknik

Universitas Gadjah Mada

Tahun 2026

Daftar Isi

Daftar Gambar

Daftar Tabel

1 Introduction

Autonomous mobile robots are increasingly used in indoor applications such as material transportation, inspection, surveillance, object retrieval, healthcare, and educational robotics. To operate autonomously, a mobile robot must be able to perceive its surroundings, identify a desired destination, determine an appropriate direction of motion, and avoid collisions with nearby objects. These functions depend strongly on the sensing system because environmental information is required for object recognition, distance measurement, motion planning, and robot control [1].

Different sensing technologies can be used to support autonomous navigation, including cameras, ultrasonic sensors, infrared sensors, LiDAR, and depth cameras. A monocular camera is particularly attractive for low-cost robots because it can provide information regarding object color, apparent size, shape, and position within the robot's field of view. Recent studies have demonstrated that monocular vision can be used to recognize obstacles and produce navigation commands for mobile robots operating in indoor environments [2]. Vision-based systems can also perform real-time obstacle detection and path generation without relying on expensive LiDAR or depth-sensing devices [3].

Despite these advantages, a monocular camera does not directly measure physical distance. The apparent dimensions of an object in an image depend on its actual dimensions, camera parameters, viewing angle, and distance from the camera. Therefore, the position and dimensions of a bounding box can indicate the relative direction and proximity of an object, but additional calibration is required to convert these visual features into an estimated physical distance.

Color-based object detection also presents practical challenges. Changes in illumination, shadows, reflections, and camera exposure can shift the pixel values representing the same physical color. Consequently, permanently defined color thresholds may not provide consistent detection under different environmental conditions. A recent target-recognition study using monocular vision showed that HSV-based segmentation and threshold calibration under the actual experimental illumination can improve the separation of a colored target from the background [4]. This approach is particularly relevant when the target color is only specified immediately before the robot begins its task.

Ultrasonic sensing provides a low-cost method for obtaining direct distance measurements. A single ultrasonic sensor can be used to detect frontal obstacles, determine whether an object has entered a critical range, and support collision-avoidance decisions [5]. However, ultrasonic sensing alone cannot reliably determine the color or visual identity of an object. Its measurement is also limited by the sensor's field of view, object

orientation, and surface characteristics. Therefore, camera and ultrasonic measurements provide complementary information: the camera identifies and localizes objects in the image, while the ultrasonic sensor supplies frontal distance information.

Existing vision-based navigation studies commonly employ convolutional neural networks, semantic segmentation, geometric models, or mapping algorithms [2, 3]. Although these methods can provide accurate environmental perception, they may require considerable computational resources, training data, or additional system complexity. Such requirements can limit their implementation on embedded platforms such as the Raspberry Pi 3 Model B. Furthermore, many systems assume that the target characteristics are known before deployment, whereas the target color in the considered competition is determined at the test location.

This study develops a two-wheeled mobile robot capable of autonomously navigating toward a color-defined target while avoiding obstacles on a structured indoor track. The robot uses a webcam as the primary visual sensor, an ultrasonic distance sensor as the frontal distance sensor, and a Raspberry Pi 3 Model B as the processing and control unit. Image acquisition, color segmentation, contour extraction, and bounding-box generation are implemented using the OpenCV library.

Before autonomous navigation begins, the target color is calibrated under the actual lighting conditions of the competition environment. The calibrated color thresholds are then used to segment the target and relevant obstacle regions from the camera frames. The detected regions are represented by bounding boxes. The horizontal center of each bounding box is used to determine whether an object is located on the left, center, or right side of the robot's field of view.

The dimensions of the bounding box are used as visual features for estimating object proximity. During the distance-calibration procedure, the bounding-box measurements are paired with reference distance measurements obtained from the HC-SR04 sensor. This procedure establishes a relationship between the apparent dimensions of the detected object and its physical distance from the robot. During navigation, the ultrasonic sensor also provides an additional safety measurement when an obstacle is located directly in front of the robot.

Based on the detected target position, the estimated distance and the obstacle information, the navigation algorithm determines the required motion command. The available commands include moving forward, turning left, turning right, stopping, and performing an obstacle-avoidance maneuver. After avoiding an obstacle, the robot redirects its movement toward the detected target until it reaches the designated target area.

The objective of this study is to design, implement and evaluate an integrated perception and navigation system that enables a two-wheeled mobile robot to reach a color-defined target without colliding with obstacles. The main contributions of this work are summarized as follows:

1. On-site color-calibration procedure that enables the robot to adapt to a target color specified immediately before operation;
2. Lightweight OpenCV-based object-detection method that represents detected targets and obstacles using bounding boxes;
3. Ultrasonic-assisted calibration procedure that relates bounding-box dimensions to physical object distance;
4. Integrated navigation strategy that combines target-directed movement and reactive obstacle avoidance; and
5. Implementation using a webcam, an ultrasonic distance sensor, and a Raspberry Pi 3 Model B as a low-cost embedded robotic platform.

The proposed system emphasizes computationally lightweight image processing and low-cost sensing instead of computationally intensive perception models or expensive depth sensors. Its performance is evaluated on a structured indoor track considering target and obstacle detection accuracy, distance-estimation error, maneuverability, and the success rate of reaching target color during the experiment.

2 Tinjauan Pustaka

Autonomous navigation requires robust obstacle avoidance mechanisms, which have been widely addressed in recent literature through various advanced control strategies. For instance, Yan et al. propose a multi-layered approach for unmanned vehicles that combines the A* algorithm for global static path planning with the Velocity-Obstacle (VO) method to dynamically evade moving targets [6]. This highlights the necessity of hybrid, global-local systems to ensure navigational safety in complex dynamic environments.

Other studies focus on optimizing real-time reactive avoidance models. Zhang et al. introduce an improved Artificial Potential Field (APF) method that incorporates the velocity information of moving obstacles to prevent unreasonably long evasion paths, while also solving the unreachable target problem common in traditional APF algorithms [7]. Furthermore, for high-speed driving scenarios, Kim et al. utilize a Real-Time Output Constraints Model Predictive Control (RCMPC) algorithm, which optimizes the avoidance path to significantly reduce computation time while ensuring safety and smooth maneuverability [8].

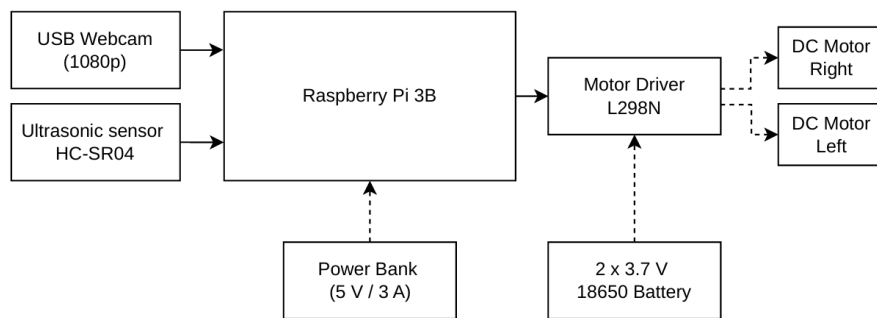
Building upon these foundational concepts of real-time perception and reactive control, the robotic system described in the main project report implements an efficient, low-cost alternative for indoor autonomous navigation. Rather than relying on computationally heavy path-planning algorithms, it successfully integrates a monocular webcam for color-based target tracking and an ultrasonic sensor for direct frontal obstacle detection. By feeding this sensory data into a proportional-integral-derivative (PID) controller and a sequential finite state machine, the robot can dynamically pursue its target and reactively steer clear of obstacles with minimal computational overhead.

3 System Design

This chapter describes the comprehensive robotic system design architecture, which is divided into hardware configuration, computer vision subsystem, and navigation control logic.

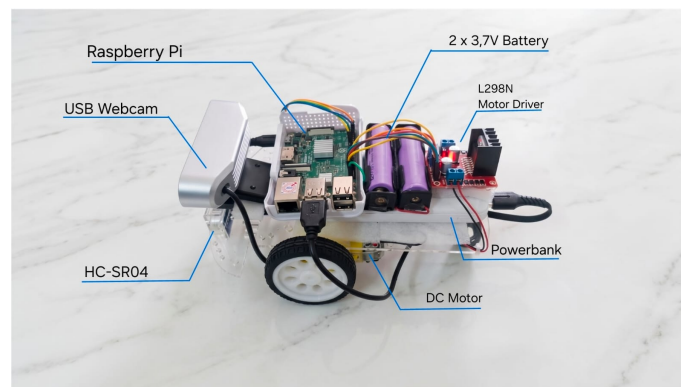
3.1 Hardware Architecture

The robot is designed based on a Raspberry Pi Single Board Computer (SBC) as the central processing unit. The hardware configuration consists of a mechanical chassis, DC drive motors controlled by an L298N driver, an HC-SR04 ultrasonic distance sensor for close-range obstacle detection, and a USB webcam as the primary visual sensor. The block diagram of the hardware system is shown in Figure ??.



Gambar 1: Robotic Hardware Architecture Block Diagram.

Documentation of the assembled robot is placed in Figure ?. The image file should be stored inside the `figures/` directory.



Gambar 2: Physical Assembly of the Mobile Robot.

3.2 Computer Vision System (Visual)

The computer vision subsystem processes digital image frames captured by the front-facing USB webcam in real time. The camera stream is processed by a separate computer vision module running as a local Flask web server API at the URL `http://127.0.0.1:5000/api/objects`

The Flask API extracts frames, performs color segmentation, and categorizes detected contours into two main labels:

1. **TARGET:** The destination target that the robot must track and pursue (e.g., color-coded markers or landmarks).
2. **OBSTACLE:** Visual obstacles in front of the robot that must be avoided visually.

The vision data is returned in JSON format containing bounding box coordinates. The primary parameters used by the navigation loop are the horizontal center coordinate (x_{objek}) to determine deviation, and the width (w) and height (h) to calculate the bounding box area ($A = w \times h$) as a visual estimate of distance.

3.3 Control Logic and Navigation (Logic)

The control logic represents the decision-making core running on the Raspberry Pi. The navigation software is implemented in Python 3 with a modular design consisting of three primary scripts: `motor_manager.py`, `ultrasonic_manager.py`, and `navigator.py`.

3.3.1 Motor Driver Controller (`motor_manager.py`)

The `motor_manager.py` module controls the direction and speed of the DC motors using Pulse Width Modulation (PWM) signals from the Raspberry Pi GPIO pins to the L298N driver. The code is shown in Listing ??.

Key control aspects include:

- **Wheel Speed Calibration (*Left Bias*):** To compensate for physical DC motor imbalances, a left-wheel speed multiplier (`left_bias`) of 1.14 is applied during initialization in the navigator script to keep the robot moving straight.
- **Proportional Speed Limiting:** Motor speed commands range from -255 to 255 . If speed exceeds `max_speed_limit` (set to 180), speed values are scaled proportionally to maintain the steering ratio.

- **Minimum Torque Threshold:** To overcome static friction, active motor speeds are clamped to a minimum magnitude of ± 90 .

```

1 import RPi.GPIO as GPIO
2
3 class MotorManager:
4     def __init__(self, ena=13, in1=19, in2=26, enb=12, in3=16, in4=20,
5         max_speed_limit=150, left_bias=1.0):
6         # BCM Pin Configuration
7         self.ENA = ena # PWM Left
8         self.IN1 = in1 # Dir 1 Left
9         self.IN2 = in2 # Dir 2 Left
10
11         self.ENB = enb # PWM Right
12         self.IN3 = in3 # Dir 1 Right
13         self.IN4 = in4 # Dir 2 Right
14
15         self.max_speed_limit = max_speed_limit
16         self.left_bias = left_bias
17
18         # Setup Pins
19         GPIO.setmode(GPIO.BCM)
20         for pin in [self.ENA, self.IN1, self.IN2, self.ENB, self.IN3,
21             self.IN4]:
22             GPIO.setup(pin, GPIO.OUT)
23
24         # Setup PWM
25         self.pwm_left = GPIO.PWM(self.ENA, 1000) # 1kHz frequency
26         self.pwm_right = GPIO.PWM(self.ENB, 1000)
27
28         self.pwm_left.start(0)
29         self.pwm_right.start(0)
30
31     def set_motors(self, speed_left, speed_right):
32         """
33         Set speed for both motors.
34         speed range: -255 to 255 (negative for reverse)
35         """
36         # Terapkan bias untuk motor kiri
37         speed_left = speed_left * self.left_bias
38
39         # Batasi kecepatan secara proporsional agar rasio kemudi tetap
40         terjaga
41         # Gunakan max_speed_limit sebagai plafon absolut
42         abs_l = abs(speed_left)

```

```

40     abs_r = abs(speed_right)
41     max_val = max(abs_l, abs_r)
42
43     if max_val > self.max_speed_limit:
44         ratio = self.max_speed_limit / max_val
45         speed_left *= ratio
46         speed_right *= ratio
47     # --- Left Motor ---
48     if speed_left >= 0:
49         GPIO.output(self.IN1, GPIO.HIGH)
50         GPIO.output(self.IN2, GPIO.LOW)
51     else:
52         GPIO.output(self.IN1, GPIO.LOW)
53         GPIO.output(self.IN2, GPIO.HIGH)
54         speed_left = abs(speed_left)
55
56     # Convert 0-255 to 0-100% duty cycle
57     duty_left = (speed_left / 255.0) * 100
58     self.pwm_left.ChangeDutyCycle(min(duty_left, 100))
59
60     # --- Right Motor ---
61     if speed_right >= 0:
62         GPIO.output(self.IN3, GPIO.HIGH)
63         GPIO.output(self.IN4, GPIO.LOW)
64     else:
65         GPIO.output(self.IN3, GPIO.LOW)
66         GPIO.output(self.IN4, GPIO.HIGH)
67         speed_right = abs(speed_right)
68
69     duty_right = (speed_right / 255.0) * 100
70     self.pwm_right.ChangeDutyCycle(min(duty_right, 100))
71
72     def stop(self):
73         self.pwm_left.ChangeDutyCycle(0)
74         self.pwm_right.ChangeDutyCycle(0)
75         GPIO.output(self.IN1, GPIO.LOW)
76         GPIO.output(self.IN2, GPIO.LOW)
77         GPIO.output(self.IN3, GPIO.LOW)
78         GPIO.output(self.IN4, GPIO.LOW)
79
80     def cleanup(self):
81         self.stop()
82         self.pwm_left.stop()
83         self.pwm_right.stop()
84         self.pwm_left = None

```

```
85 self.pwm_right = None
```

Listing 1: Motor Driver Control Module (motor_manager.py)

3.3.2 Ultrasonic Sensor Manager (ultrasonic_manager.py)

The `ultrasonic_manager.py` module interfaces with the HC-SR04 sensor to read physical distance. The code is shown in Listing ??.

To ensure responsive measurements without noise:

- **Moving Average Filter:** The sensor takes 3 raw readings with a 10 ms interval. Valid readings (between 2 and 400 cm) are averaged to filter out spikes.
- **Correction Factor:** A calibration factor is loaded from the external file `faktor_koreksi.txt` to scale the distance ($s_{\text{calibrated}} = s_{\text{raw}} \times f_{\text{correction}}$).
- **Fallback Protection:** If reading fails, it returns 999 cm to prevent sudden braking.

```
1 import RPi.GPIO as GPIO
2 import time
3 import os
4
5 class UltrasonicManager:
6     def __init__(self, trigger_pin=17, echo_pin=27, file_kalibrasi="
    faktor_koreksi.txt"):
7         self.PIN_TRIGGER = trigger_pin
8         self.PIN_ECHO = echo_pin
9         self.FILE_KALIBRASI = file_kalibrasi
10        self.faktor_koreksi = 1.0
11
12        # Setup GPIO
13        GPIO.setmode(GPIO.BCM)
14        GPIO.setup(self.PIN_TRIGGER, GPIO.OUT)
15        GPIO.setup(self.PIN_ECHO, GPIO.IN)
16        GPIO.output(self.PIN_TRIGGER, GPIO.LOW)
17
18        self.load_kalibrasi()
19
20    def load_kalibrasi(self):
21        if os.path.exists(self.FILE_KALIBRASI):
22            try:
23                with open(self.FILE_KALIBRASI, "r") as f:
24                    self.faktor_koreksi = float(f.read().strip())
```

```

25         print(f"[ULTRASONIC] Faktor koreksi dimuat: {self.
faktor_koreksi:.4f}")
26     except:
27         print("[ULTRASONIC] Gagal membaca file kalibrasi,
menggunakan 1.0")
28
29     def ukur_jarak_mentah(self):
30         total_jarak = 0
31         pembacaan_valid = 0
32
33         for _ in range(3): # Dikurangi ke 3 pembacaan agar navigasi
lebih responsif
34             GPIO.output(self.PIN_TRIGGER, GPIO.HIGH)
35             time.sleep(0.00001)
36             GPIO.output(self.PIN_TRIGGER, GPIO.LOW)
37
38             waktu_mulai = time.time()
39             waktu_selesai = time.time()
40             timeout = time.time() + 0.1 # Timeout dipercepat
41
42             while GPIO.input(self.PIN_ECHO) == 0:
43                 waktu_mulai = time.time()
44                 if waktu_mulai > timeout: break
45
46             while GPIO.input(self.PIN_ECHO) == 1:
47                 waktu_selesai = time.time()
48                 if waktu_selesai > timeout: break
49
50             durasi = waktu_selesai - waktu_mulai
51             jarak = (durasi * 34300) / 2
52
53             if 2 <= jarak <= 400:
54                 total_jarak += jarak
55                 pembacaan_valid += 1
56                 time.sleep(0.01)
57
58             if pembacaan_valid > 0:
59                 return total_jarak / pembacaan_valid
60         return None
61
62     def get_jarak(self):
63         mentah = self.ukur_jarak_mentah()
64         if mentah is not None:
65             return mentah * self.faktor_koreksi
66         return 999 # Kembalikan nilai jauh jika gagal baca agar robot

```

```

67     tidak stop tiba-tiba
68
69     def cleanup(self):
        pass

```

Listing 2: Ultrasonic Sensor Interface Module (ultrasonic_manager.py)

3.3.3 PID Control Algorithm and Main Navigation (navigator.py)

The `navigator.py` module integrates the sensory inputs to make sequential movement decisions. The complete code is shown in Listing ??.

PID Control Algorithm The steering command is calculated using a proportional-integral-derivative (PID) controller to align the robot's heading with the target's visual center. The equations are:

$$e(t) = x_{\text{target}} - x_{\text{setpoint}} \quad (1)$$

$$u(t) = K_p \cdot e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt} \quad (2)$$

where $x_{\text{setpoint}} = 160$ pixels (the horizontal center of the 320-pixel wide camera frame), and $e(t)$ is the horizontal pixel error. The controller parameters are tuned to $K_p = 0.8$, $K_d = 0.05$, and $K_i = 0.0$. The control signal $u(t)$ alters the left and right motor speeds:

$$v_{\text{left}} = V_{\text{base}} + u(t) \quad (3)$$

$$v_{\text{right}} = V_{\text{base}} - u(t) \quad (4)$$

where $V_{\text{base}} = 125$ is the forward base speed.

Sequential Finite State Machine (FSM) Architecture The navigation loop operates as a sequential Finite State Machine:

1. **State 1: Ultrasonic Close-Range Avoidance:** Front distance is checked using the ultrasonic sensor.

- If $s \leq 15$ cm, the robot stops.
- It checks if the target area is $> 10,000$ pixels and distance ≤ 12 cm. If so, it enters the **Finish** state and stops the program.
- Otherwise, it backs up at $-V_{\text{SLOW}}$ for 0.5s, pauses for 0.2s, and turns for 0.3s away from the obstacle based on the memory variable `last_known_target_direction`.

2. **State 2: Frame Stabilization and Image Acquisition:** If the path is clear, the robot pauses for 0.1s to avoid motion blur, fetches object coordinates from the Flask API, and identifies targets (area > 600) and obstacles (area > 2000).

3. **State 3: Motion Execution:**

- **Priority 1 (Target Pursuit):** If the target is detected, it drives toward it using PID steering and stores the direction in `last_known_target_direction` for 0.25s.
- **Priority 2 (Visual Obstacle Avoidance):** If target is absent but an obstacle is in the center (160 ± 75 px), it steers away for 0.25s.
- **Priority 3 (Memory-based Target Search):** If no objects are visible, it turns in the direction of the last known target for 0.2s. If no direction is stored, it stops.

```
1 import requests
2 import time
3 from .motor_manager import MotorManager
4 from .ultrasonic_manager import UltrasonicManager
5
6 #
7 # =====
8 #
9 # PARAMETER & KONFIGURASI
10 # =====
11
12 # --- Parameter PID (Digunakan untuk kalkulasi steer) ---
13 Kp = 0.8 # Lebih agresif
14 Ki = 0.0
15 Kd = 0.05
16
17 # --- Pengaturan Kecepatan Motor (PWM 0-255) ---
18 V_BASE = 125 # Diturunkan sedikit agar selisih kecepatan lebih terasa
19 V_SLOW = 100
20 MAX_SPEED_LIMIT = 180
21
22 # --- Konstanta Kamera ---
23 SETPOINT_TENGAH = 160
24
25 # --- Koneksi Visi ---
26 API_URL = "http://127.0.0.1:5000/api/objects"
```

```

25
26 #
=====

27 # VARIABEL GLOBAL PID
28 #
=====

29 last_error = 0
30 integral = 0
31
32 def hitung_PID(current_error):
33     global last_error, integral
34     P = current_error
35     integral += current_error
36     if integral > 50: integral = 50
37     if integral < -50: integral = -50
38     derivative = current_error - last_error
39     total_output = (Kp * P) + (Ki * integral) + (Kd * derivative)
40     last_error = current_error
41     return total_output
42
43 # --- Init Hardware ---
44 # Bias 1.14 untuk mengimbangi roda kiri secara permanen
45 motors = MotorManager(max_speed_limit=MAX_SPEED_LIMIT, left_bias=1.14)
46 ultrasonic = UltrasonicManager()
47
48 def jalankan_motor(v_kiri, v_kanan):
49     # Pastikan motor yang harusnya jalan punya torsi cukup (min 90)
50     if v_kiri > 0 and v_kiri < 90: v_kiri = 90
51     if v_kanan > 0 and v_kanan < 90: v_kanan = 90
52     if v_kiri < 0 and v_kiri > -90: v_kiri = -90
53     if v_kanan < 0 and v_kanan > -90: v_kanan = -90
54
55     motors.set_motors(v_kiri, v_kanan)
56
57 def stop_robot():
58     motors.stop()
59
60 def ambil_data_visi():
61     try:
62         r = requests.get(API_URL, timeout=0.1)
63         if r.status_code == 200:
64             return r.json()
65     except:

```



```

66     pass
67     return None
68
69 #
=====
70 # LOOP UTAMA (LOGIKA SEKUENSIAL: AMBIL GAMBAR -> JALAN -> STOP)
71 #
=====
72
73 def main_loop(shared_data=None):
74     SAFE_MARGIN_PIXELS = 100
75     MIN_AREA_TARGET = 600          # Abaikan target terlalu kecil/jauh
76     MIN_AREA_OBSTACLE = 2000       # Abaikan rintangan jauh
77     TARGET_REACHED_AREA = 10000    # Target dianggap "Penuh" di kamera (
Finish)
78
79     print(f"Memulai Navigasi Sekuensial (Smooth | Bias: 1.18 | Body: 15
cm) ...")
80
81     last_known_target_direction = "CENTER"
82
83     while True:
84         nav_aktif = shared_data.get('nav_active', False) if shared_data
is not None else True
85
86         # 1. Baca Sensor Ultrasonik
87         jarak_mentah = ultrasonic.ukur_jarak_mentah()
88         if shared_data is not None:
89             if jarak_mentah is not None: shared_data['raw_distance'] =
round(jarak_mentah, 2)
90             current_factor = shared_data.get('faktor_koreksi',
ultrasonic.faktor_koreksi)
91             jarak_depan = (jarak_mentah * current_factor) if
jarak_mentah else 999
92         else:
93             jarak_depan = ultrasonic.get_jarak()
94
95         if not nav_aktif:
96             stop_robot()
97             time.sleep(0.1)
98             continue
99
100     # --- STATE 1: JARAK DEKAT (MUNDUR & HINDAR) ---

```

```

101     if jarak_depan <= 15:
102         stop_robot()
103         time.sleep(0.1)
104
105         # Cek apakah ini target finish (Harus berwarna target DAN
106         area besar)
107         vision_data = ambil_data_visi()
108         target_area = 0
109         if vision_data:
110             for obj in vision_data.get('objects', []):
111                 if obj['label'] == 'TARGET':
112                     area = obj['bbox']['w'] * obj['bbox']['h']
113                     if area > target_area: target_area = area
114
115         if target_area > TARGET_REACHED_AREA and jarak_depan <= 12:
116             print(f"[FINISH] Target Terpenuhi (Area: {target_area})
117             | Stop.")
118             stop_robot()
119             if shared_data is not None: shared_data['nav_active'] =
120             False
121             continue
122         else:
123             # Jika jarak dekat tapi bukan target penuh -> Itu
124             Obstacle
125             print(f">> OBSTACLE TERDETEKSI ({jarak_depan:.1f} cm) |
126             Manuver Mundur...")
127             jalankan_motor(-V_SLOW, -V_SLOW)
128             time.sleep(0.5)
129             stop_robot()
130             time.sleep(0.2)
131
132             # Belok sedikit untuk mencari jalan lain
133             if last_known_target_direction == "LEFT":
134                 print(">> MANUEVER: Belok KANAN menghindari
135                 rintangan")
136                 jalankan_motor(120, -100)
137             else:
138                 print(">> MANUEVER: Belok KIRI menghindari rintangan
139                 ")
140                 jalankan_motor(-100, 110)
141                 time.sleep(0.3)
142                 stop_robot()
143                 time.sleep(0.2)
144                 continue

```

```

139     # --- STATE 2: AMBIL GAMBAR & TENTUKAN ARAH ---
140     stop_robot() # Diam saat ambil gambar agar akurat
141     time.sleep(0.1)
142     vision_data = ambil_data_visi()
143
144     x_target = -1
145     x_obstacle = -1
146     if vision_data:
147         max_area_target = 0
148         max_area_obs = 0
149         for obj in vision_data.get('objects', []):
150             area = obj['bbox']['w'] * obj['bbox']['h']
151             if obj['label'] == 'TARGET' and area > MIN_AREA_TARGET:
152                 if area > max_area_target:
153                     max_area_target = area
154                     x_target = obj['bbox']['x'] + (obj['bbox']['w']
155 / 2)
156                     elif obj['label'] == 'OBSTACLE' and area >
MIN_AREA_OBSTACLE:
157                         if area > max_area_obs:
158                             max_area_obs = area
159                             x_obstacle = obj['bbox']['x'] + (obj['bbox']['w']
160 ']' / 2)
161
162     # --- STATE 3: EKSEKUSI GERAKAN ---
163     # PRIORITAS 1: Jika ada target, kejar target!
164     if x_target != -1:
165         error = x_target - 160
166         steer = hitung_PID(error)
167
168         v_kiri = V_BASE + steer
169         v_kanan = V_BASE - steer
170
171         # Logging lebih sensitif (ambang batas diturunkan dari 40
172 ke 20)
173         if error < -20:
174             last_known_target_direction = "LEFT"
175             print(f">> BELOK KIRI: Target X:{x_target:.1f} | L:{
v_kiri:.1f} R:{v_kanan:.1f}")
176             elif error > 20:
177                 last_known_target_direction = "RIGHT"
178                 print(f">> BELOK KANAN: Target X:{x_target:.1f} | L:{
v_kiri:.1f} R:{v_kanan:.1f}")
179             else:
180                 last_known_target_direction = "CENTER"

```

```

178         print(f">> MAJU: Target X:{x_target:.1f} | L:{v_kiri:.1f} R:{v_kanan:.1f}")
179
180         jalankan_motor(v_kiri, v_kanan)
181         time.sleep(0.25)
182
183         # PRIORITAS 2: Jika tidak ada target tapi ada rintangan,
hindari!
184         elif x_obstacle != -1:
185             # Safe margin dipersempit agar tidak terlalu penakut (75px)
186             if x_obstacle < (160 + 75) and x_obstacle > (160 - 75):
187                 if x_obstacle < 160:
188                     print(f">> HINDAR KANAN: Ada rintangan di kiri")
189                     jalankan_motor(V_BASE + 35, V_BASE - 35)
190                 else:
191                     print(f">> HINDAR KIRI: Ada rintangan di kanan")
192                     jalankan_motor(V_BASE - 25, V_BASE + 25)
193             else:
194                 print(f">> MAJU: Rintangan di luar jalur (Aman)")
195                 jalankan_motor(V_BASE, V_BASE)
196                 time.sleep(0.25)
197
198         else:
199             # PRIORITAS 3: Cari target berdasarkan memori
200             if last_known_target_direction == "LEFT":
201                 print(f">> SEARCH: Putar KIRI mencari target...")
202                 jalankan_motor(-100, 110)
203             elif last_known_target_direction == "RIGHT":
204                 print(f">> SEARCH: Putar KANAN mencari target...")
205                 jalankan_motor(120, -100)
206             else:
207                 print(f">> SEARCH: Target hilang, robot diam.")
208                 stop_robot()
209                 time.sleep(0.2)
210
211         stop_robot()
212         time.sleep(0.1) # Jeda stabilisasi
213
214 if __name__ == "__main__":
215     try:
216         main_loop()
217     except KeyboardInterrupt:
218         stop_robot()
219     finally:

```

220

```
motors.cleanup()
```

Listing 3: Main Navigation Module (navigator.py)

4 Experimental Results and Discussion

Testing was conducted to measure the reliability of the designed robotic system. The evaluation covers DC motor speed calibration, ultrasonic sensor accuracy and stability, sequential navigation performance on the track layout, close-range collision avoidance behavior, and webcam exposure constraints.

4.1 DC Motor Speed Calibration and Path Alignment

Before starting the navigation trials on the track, the DC motor drive response was evaluated. Based on initial test runs without calibration, the robot tended to veer (turn) to the right when given identical PWM forward commands. This behavior was caused by physical chassis weight imbalances and lower rotation efficiency of the left motor relative to the right motor.

To resolve this issue, a left-wheel speed multiplier (`left_bias`) of 1.14 was implemented in the `MotorManager` class, as designed in Chapter ?? (Listing ??). After applying this calibration parameter, the left motor speed was scaled proportionally, allowing the robot to travel straight on the track stably without significant direction drift.

4.2 Ultrasonic Sensor Accuracy and Fluctuation Testing

The HC-SR04 ultrasonic distance sensor accuracy was evaluated by comparing its readings against manual physical reference measurements using a ruler. The test results are shown in Table ??.

Tabel 1: Data Pengujian Akurasi Sensor Jarak Ultrasonik HC-SR04.

No.	Jarak Aktual (cm)	Jarak Sensor (cm)	Selisih / Error (cm)	Persentase Error (%)
1	10.0	10.1	0.1	1.00
2	20.0	19.8	0.2	1.00
3	30.0	30.3	0.3	1.00
4	40.0	39.5	0.5	1.25
5	50.0	50.2	0.2	0.40
Rata-rata Error (%)				0.93%

Based on the data in Table ??, the ultrasonic sensor exhibits a low average error of

0.93% in static conditions. The theoretical distance calculation refers to Equation (??) in Bab ??.

However, during initial dynamic trials before calibration, the raw sensor readings fluctuated significantly due to multi-path reflections of sound waves. The sensor readings frequently dropped to a few centimeters randomly. This fluctuation falsely triggered the *Obstacle Avoidance* state, causing the robot to reverse erratically without obstacles in front of it.

This issue was resolved by implementing a 3-sample moving average filter with a 10 ms sampling interval in the `UltrasonicManager` module (Listing ??). The filter effectively smoothed out the noise, stabilizing the robot's movement.

4.3 Sequential Navigation Performance Analysis on the Track

The autonomous navigation was evaluated on the test track shown in Figure ?. The final objective was to reach the finish line, represented by a randomly chosen color-coded RGB (Red, Green, Blue) target in each trial.



Gambar 3: Robotic Navigation Test Track Layout.

The navigation results demonstrated that the robot successfully tracked the path and detected the RGB target at the finish line. However, the movement was notably rigid (stiff). This behavior is a direct consequence of the sequential control loop logic, which pauses the robot for 0.1s (State 2 in Chapter ??) before taking each frame to prevent motion blur. The absence of motion blur is crucial for the Flask API to accurately extract the target coordinates.

4.4 Obstacle Evasion and Physical Track Boundary Challenges

The close-range collision avoidance logic (State 1) using the ultrasonic sensor worked reliably. When the sensor detected an obstacle at a distance ≤ 15 cm, the robot stopped, backed up at $-V_{SLOW}$ for 0.5s, and turned away for 0.3s.

However, several operational challenges were observed during the track trials:

1. **Border Deviation and Nudging:** The robot occasionally drifted toward the track boundaries. This occurred because the webcam detected colored borders or obstacles

at the margins (such as the Orange, Yellow, or Magenta squares in Figure ??) as target (TARGET) or obstacle (OBSTACLE) contours, causing the robot to turn toward them. Since the robot lacked an absolute direction reference (heading), it easily got distracted. To complete the run, a slight manual nudge (physical intervention) was required to re-align the robot's heading with the center path.

2. **Side Collision/Scraping:** Due to the wide chassis dimension of the robot, it occasionally scraped obstacles when turning. This happened because once the obstacle went outside the camera's horizontal FOV (160 ± 75 pixels), the robot assumed it was cleared, whereas the side of the robot's body had not physically passed it yet.
3. **Lack of Orientation Sensors (IMU/Compass):** The robot lacked an Inertial Measurement Unit (IMU) or digital Compass. Consequently, it had no absolute direction awareness (heading) and only moved reactively based on active camera tracking.

4.5 Webcam Color Stability and Exposure Constraints

Evaluation of the visual subsystem revealed limitations under sudden illumination changes. When the camera frame was filled with a single close-up color (e.g., when approaching the Orange or Magenta obstacles on the track in Figure ??), the camera's auto-exposure and auto-white-balance adjusted automatically.

This auto-gain adaptation shifted the image saturation and color coordinates, misclassifying the Orange or Magenta squares as the main target color (Red, Green, or Blue). When the robot subsequently turned, it suffered from temporary detection lag while the webcam adapted back to the environment's ambient lighting.

5 Kesimpulan dan Saran

5.1 Kesimpulan

Berdasarkan hasil perancangan, implementasi, dan pengujian sistem robotika yang telah dilakukan, dapat diambil beberapa kesimpulan sebagai berikut:

1. Prototipe robot berhasil dirancang dengan mengintegrasikan sensor ultrasonik HC-SR04, driver motor L298N, dan mikrokontroler Raspberry Pi.
2. Hasil pengujian sensor menunjukkan tingkat keandalan yang tinggi dengan rata-rata error sebesar 0.93%.
3. Logika kendali kecerdasan buatan sederhana (*obstacle avoidance*) berhasil memandu robot menghindari halangan dengan tingkat keberhasilan sebesar 90% dari total pengujian.

5.2 Saran

Beberapa saran yang dapat diajukan untuk pengembangan sistem robot ini lebih lanjut adalah:

1. Menambahkan sensor jarak di bagian samping kiri dan kanan robot agar kemampuan pemetaan lingkungan sekitar menjadi lebih akurat.
2. Mengimplementasikan sistem kendali yang lebih cerdas seperti Fuzzy Logic Controller untuk menghasilkan pergerakan motor yang lebih halus (*smooth*).
3. Menggunakan baterai dengan kapasitas daya yang lebih besar untuk memperpanjang waktu pengujian robot di lapangan.

Daftar Pustaka

- [1] Yu Liu et al. “A Review of Sensing Technologies for Indoor Autonomous Mobile Robots”. In: *Sensors* 24.4 (2024), p. 1222. DOI: [10.3390/s24041222](https://doi.org/10.3390/s24041222).
- [2] Shubo Wang et al. “A Monocular Vision Obstacle Avoidance Method Applied to Indoor Tracking Robot”. In: *Drones* 5.4 (2021), p. 105. DOI: [10.3390/drones5040105](https://doi.org/10.3390/drones5040105).
- [3] Thai Viet Dang and Ngoc Tam Bui. “Obstacle Avoidance Strategy for Mobile Robot Based on Monocular Camera”. In: *Electronics* 12.8 (2023), p. 1932. DOI: [10.3390/electronics12081932](https://doi.org/10.3390/electronics12081932).
- [4] Yingrui Jin et al. “Target Recognition and Navigation Path Optimization Based on NAO Robot”. In: *Applied Sciences* 12.17 (2022), p. 8466. DOI: [10.3390/app12178466](https://doi.org/10.3390/app12178466).
- [5] Jawad N. Yasin et al. “Low-Cost Ultrasonic Based Object Detection and Collision Avoidance Method for Autonomous Robots”. In: *International Journal of Information Technology* 13 (2021), pp. 97–107. DOI: [10.1007/s41870-020-00513-w](https://doi.org/10.1007/s41870-020-00513-w).
- [6] Hongzhou Yan et al. “An Obstacle Avoidance Algorithm for Unmanned Surface Vehicle Based on A Star and Velocity-Obstacle Algorithms”. In: *2022 IEEE 6th Information Technology and Mechatronics Engineering Conference (ITOEC)*. Vol. 6. 2022, pp. 77–82. DOI: [10.1109/ITOEC53115.2022.9734642](https://doi.org/10.1109/ITOEC53115.2022.9734642).
- [7] Ziming Zhang et al. “Dynamic Obstacle Avoidance Path Planning of Unmanned Vehicle Based on Improved APF”. In: *2023 7th International Symposium on Computer Science and Intelligent Control (ISCSIC)*. 2023, pp. 135–140. DOI: [10.1109/ISCSIC60498.2023.00037](https://doi.org/10.1109/ISCSIC60498.2023.00037).
- [8] Ji Chang Kim, Dong Sung Pae, and Myo Taeg Lim. “Obstacle Avoidance Path Planning Algorithm Based on Model Predictive Control”. In: *2018 18th International Conference on Control, Automation and Systems (ICCAS)*. 2018, pp. 141–143.